

The background of the entire image is a dark blue field filled with a pattern of red dots. These dots are arranged in a way that they form a large, faint circular shape in the center, with the density of the dots being higher in the center and fading towards the edges. The dots are of varying sizes, creating a textured, digital effect.

HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

HEALTHCARE BIG DATA ANALYTICS SYSTEM

Group 8:

Pham Quang Huy - 20252271M, Nguyen Thi Mai Anh - 20252265M
Tran Duy Khanh - 20252266M, Duong Hoang Hai - 20252273M
Do Hoang Viet - 20252744M

ONE LOVE. ONE FUTURE.

Why Healthcare?

- ▶ The healthcare sector is undergoing rapid **digital transformation**
- ▶ Medical data is growing quickly in **scale, velocity, and complexity**
- ▶ Leveraging and integrating healthcare data enables: Improvement of operational efficiency, Optimization of resource allocation and Enhancement in patient care quality

Main Objectives:

- ▶ Simulate a **Big Data system** for healthcare
- ▶ Build an end-to-end data pipeline:
Ingestion → Cleaning → Storage → Analytics → Visualization
- ▶ Enable practical applications for **healthcare decision support**

Objective 1: Data Collection

- ▶ Integrate ≥ 3 **healthcare datasets**: patient, insurance, lab results, healthcare services

Objective 2: Data Storage. Build a multi-layer **Data Lake architecture** on **MinIO**, organizing data from Raw $\rightarrow$ Bronze $\rightarrow$ Silver $\rightarrow$ Gold.

Objective 3: Data Processing. Develop data processing pipelines using **Apache Spark**, supporting both batch (historical) and streaming (real-time) processing.

Objective 4: Analytics & Visualization. Perform analytics and visualization using **Apache Superset** to enable trend detection, risk prediction, and cost analysis.

The Big Data system consists of 4 layers:

1. **Deployment Layer:** Docker Compose
 2. **Storage Layer:** MinIO (Delta Lake) & MongoDB
 3. **Processing Layer:** Apache Spark & Apache Kafka
 4. **Analytics Layer:** Apache Superset
- ▶ Combines **Batch Processing** and **Streaming Processing**
 - ▶ All storage layers use **Delta Lake** format (ACID + Time Travel)
 - ▶ Ensures flexible storage and real-time responsiveness

Healthcare Datasets

Dataset	Key Fields	Data Types	Description
Patients	Name, Age, Gender, Blood Type, Medical Condition, Admission/Discharge, <i>etc.</i>	String / Integer / Double / Date	Detailed patient records, clinical info, treatment costs
Insurance	Age, Sex, BMI, Children, Smoker, Region, Charges, <i>etc.</i>	Integer / String / Double	Insurance cost and demographic data
Diabetes	Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, Age, Outcome, <i>etc.</i>	Integer / Double / Boolean	Biometrics and diabetes diagnosis
NoShows	PatientID, AppointmentID, Gender, ScheduledDay, AppointmentDay, <i>etc.</i>	String / Integer / Boolean / Date	Appointment behavior and no-show results

RAW → BRONZE: Ingestion & Quality Gate

Step	Description	Tools
Schema Enforcement	Read CSV with predefined schema; reject type mismatches at ingestion time	StructType, HealthcareSchemas
Column Normalization	Convert names to snake_case (Blood Type → blood_type); required by Delta Lake	regexp_replace
Basic Dedup & Null Drop	Remove fully-duplicate rows and all-null rows	dropDuplicates(), na.drop()
DQ Check + Quarantine	Flag rows violating rules (invalid age, negative billing, etc.); route bad rows to quarantine zone instead of dropping silently	DQChecker, Delta append
Metadata & Write	Add channel (e.g. "hospital") and load_date; write each dataset as an independent Delta table	lit(), current_date(), Delta

RAW → BRONZE: Schema Enforcement

- ▶ Schema enforcement ensures data follows a **predefined structure and types** at ingestion time, applied to all Bronze datasets (patients, insurance, diabetes, noshows)
- ▶ **Benefits:** prevents missing fields, catches malformed records early, and blocks accidental schema changes on every Delta Lake write

```
# Row PASS – schema enforcement: room_number → null nhưng vẫn vào Bronze
"Alice TypeEnforce,45,Female,A+,Hypertension,2024-01-15,"
"Dr. Smith,City Hospital,Aetna,5000.50,INVALID_ROOM,"
"Elective,2024-01-20,Aspirin,Normal\n"
```

name	age	room_number	billing_amount
Alice TypeEnforce	45	NULL	5000.5

Figure: Schema Enforcement

CDP-Based Data Integration Model

Each **patient** is treated as a *customer*; each dataset is a separate **data channel**:

CDP Concept	Dataset	Information Captured
Customer / Channel 1 – Hospital	patients	Treatment history, billing, admission
Channel 2 – Insurance	insurance	BMI, insurance charges, smoker status
Channel 3 – Lab Results	diabetes	Glucose, insulin, diabetes outcome
Channel 4 – Appointment	noshows	Appointment history, show/no-show rate

Challenge: 4 datasets from independent systems — no shared `patient_id`

Solution: Aggregate each channel by age + gender before joining

- ▶ Avoids Cartesian explosion: $55,000 \times 1,300 \approx 71\text{M}$ rows
- ▶ Silver layer result: **55,500 rows** — exactly 1 Unified Profile per patient

BRONZE → SILVER: Normalization & Data Integration

Step	Description	Tools / Techniques
Schema normalization	Rename inconsistent columns (Age → age, Gender → gender, BMI → bmi, etc.); add <code>name_norm</code> (lowercase, clean text)	<code>withColumnRenamed</code> , <code>lower</code> , <code>regexp_replace</code>
Data integration (join)	Aggregate each channel by age/gender, then left-join onto patients to create a Unified Patient Profile	<code>groupBy</code> , <code>agg</code> , <code>join()</code>
Data quality control	Ensure data consistency, no domain violations, removal of duplicates	<code>DQChecker</code> , <code>dropDuplicates</code>
Write to Silver	Write unified dataset partitioned by dataset, <code>load_date</code> as Delta table	<code>write.format("delta").partitionBy(...)</code>

All 3 layers (Bronze / Silver / Gold) use Delta Lake format:

- ▶ **ACID transactions** — partial writes are automatically rolled back on failure
- ▶ **Schema Enforcement** — Delta rejects writes that violate the existing schema
- ▶ **Time Travel** — every write is recorded as a new version; any snapshot is queryable

Querying historical versions:

Query	Syntax
By version number	<code>.option("versionAsOf", 0)</code>
By timestamp	<code>.option("timestampAsOf", "2025-10-26")</code>
View full history	<code>DeltaTable.forPath(...).history().show()</code>
Restore a version	<code>RESTORE TABLE delta.`path` TO VERSION AS OF 0</code>

Use case: If incorrect data is written to Silver, restore to the previous version without rerunning the entire pipeline.

Overall Processing Pipeline

Step	Description	Tools / Functions
Read Silver Layer	Load unified profile from s3a://<bucket_silver>/data/	<code>spark.read.format("delta")</code>
Filter unified data	Keep only dataset = 'healthcare_unified'	<code>.where("dataset='healthcare_unified'")</code>
Compute KPIs	Average insurance cost (avg_insurance_charge) and treatment cost (billing_amount) grouped by gender	<code>groupBy("gender").agg(avg(...))</code>
Write to Gold Layer	Write KPI table (2 rows: Female / Male) partitioned by dataset, load_date	<code>write.format("delta").partitionBy(...).save()</code>

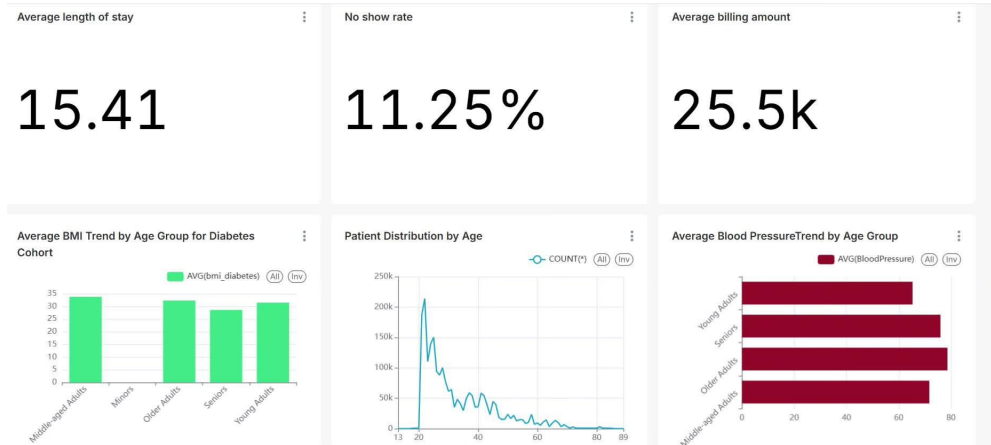


Figure: Example of Data Analytics

Streaming pipeline runs in parallel with the batch pipeline:

1. **Producer** — Python script generates mock healthcare events (e.g. new visits, lab results) and publishes to Kafka topic `healthcare_events`
2. **Spark Structured Streaming** — consumes Kafka topic in micro-batches, parses JSON payload, applies enrichment
3. **MongoDB** — processed records are written to collection `healthcare.patient_stream` for real-time access

Component	Detail
Kafka cluster	3 brokers, ports 19092 / 29092 / 39092
Spark checkpoint	Local <code>./checkpoints/</code> (fault tolerance)
MongoDB sink	<code>healthcare.patient_stream</code> collection



HUST

THANK YOU !

